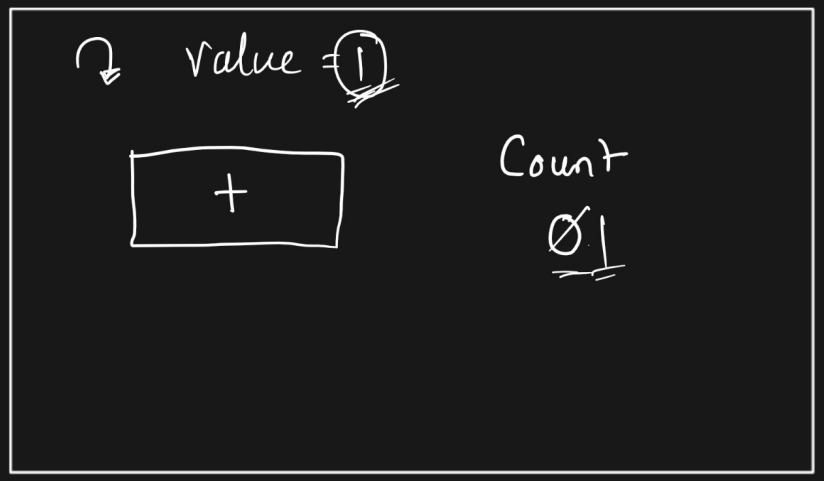
useRef

use Case

A)



+ → Click

onClick

↓ handle

handlerFun?

↓ value + 1

setCount (count + 1)

↓

update (count increment)

↓

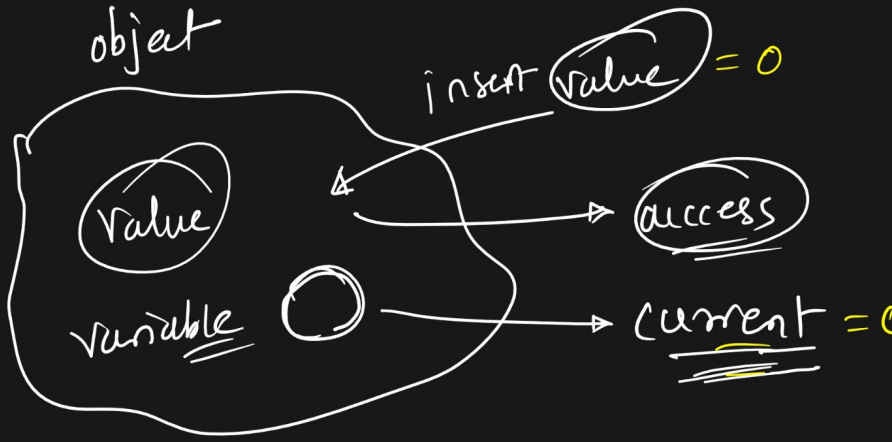
component rerender

value = 2 → value does not persist during re rendering

current

↓

useRef (0)



useRef hook → return → object

(we persist the local variable value across rerendering)

let value = useRef(0)

State Variable

count

Vs

ref variable

value . current

when State variable will be update then our component will be rerender

when ref variable will be change that time we can not rerender page.

B) useRef → use → to access other html component directly

→ DOM → element → access

e.g.



Count

0

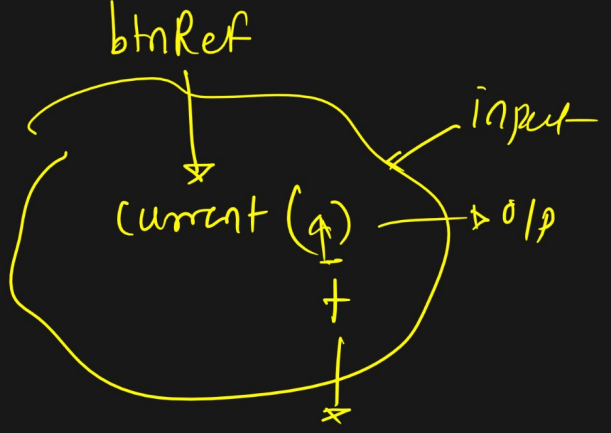


→ click

object

let btnRef = useRef()

ref = { btnRef } > + <



Style: bc
↓
red

useMemo :-

hook Fun?

Memo
↳ Memoization

↓

prevent

↓

Unnecessary task prevent by useMemo

Component

(A) (B) ↓ Fun?

4×3 → 4×3 = 12 store

2×6 → 2×6 = 12 store

6×6 → 6×6 = (36) store

6×6 → 6×6 = 36 X

Fun?

+ → onClick

→ handler → setCount(+1)

↳ page rerender

↳ input = 1

22

entTask (1) → execute value 2

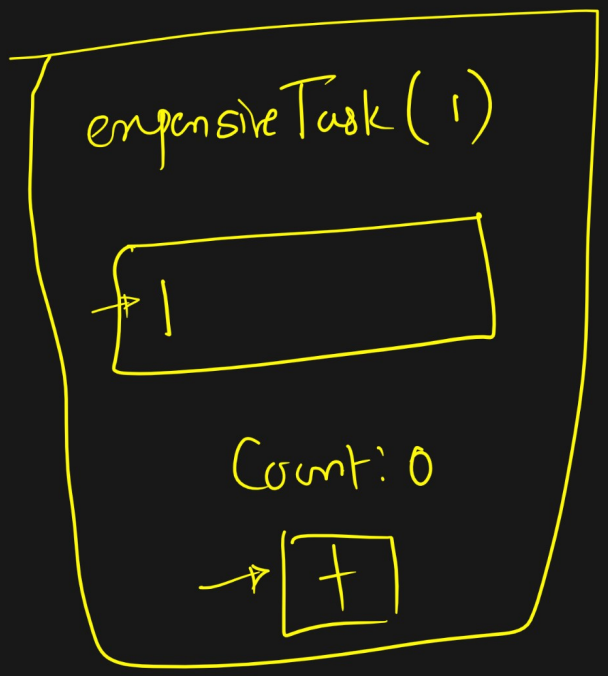
Count: 0

f

↳ fn(

{

}



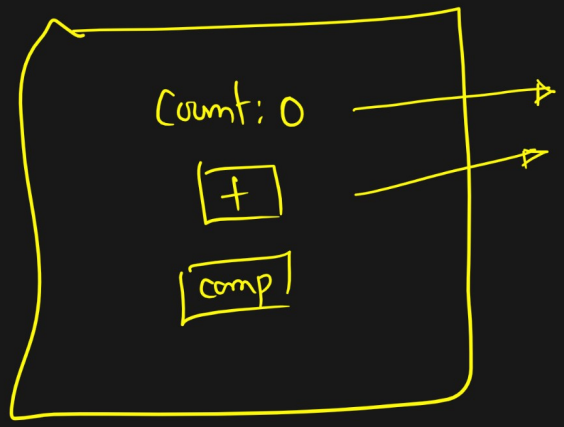
+ | +2 | Rerender

↓

expensiveTask (x/2) → Calculate

→ result → Store

useCallback :- Used to prevent the recreation of fun?



click → onClick → handler

↓ count

setCount(+)

↓

rerender

↓

handler fun? → ref? → re-create

re-execute

prevent → useCallback

useMemo → prevent re-execute

useMemo vs useCallback

Rerender
fun^n
prevent
for
reexecute
expensive

Rerender
fun^n — ref → prevent
recreation

Revision of → React

1) objective / use React
2) create html ele - JS/react
3) import, export
4) Real DOM & virtual DOM
5) JSX
6) How to make a react project
7) Folder structure
8) vite → Bundler
9) rafce → snippet

JS

ES6
1) Execution context
2) variable Decl^r — let, const, var
3) hoisting — variable
4) data types
5) fun^n def^n — ways
6) fun^n ≠ hoisting
7) Fun^n types → higher order fun
   callback fun^n
8) array → fun^n

10) props
11) multiple component
12) props drilling
13) Destructure the props
14) state
15) state management
16) hooks
17) useState
18) State lifting
19) Condn rendering
20) Event handling
21) useEffect
22) useContext
23) React Routing
24) useProvider
25) Nested Routing
26) useForm Hook
27) React Redux Toolkit

map, filter, reduce, forEach
9) string - string op
10) object
11) freeze, seal
12) deep copy, shallow copy
13) Rest & Param op
14) Destructure the object
15) Asynchronous Prog.
16) setTimeout
17) clearTimeout
18) setInterval
19) clearInterval
20) Issues of callback fun
21) Event loop
22) callback queue
23) Call stack
24) Promises
25) Async Await

useDispatch

28) useRef

29) useMemo

30) useCallback